



Compte rendu

Recherche des zones blanches

Eliot CALDERON Y MORA

Table des matières

Ressources	3
Changements	4
Collecte des antennes	4
Données	4
Analyse des antennes	5
generer_antenne_file	5
readCSVLineAntenne	5
Recherches des zones blanches	6
Vocabulaire	6
Algorithme	6
Version simplifiée	7
Itinéraire	7
find_dead_zone	7
Exécution	7
Version avancée	8
Itinéraires	8
Zones blanches	9
Fichier de retour	11
Système d’affichage web	11
Manuel d’utilisation version avancée	12
Récupération des antennes	12
Téléchargement des données	12
Emplacement du fichier	12
Exécution	12
Trouver les zones blanches	13
Emplacement du fichier	13
Exécution	13
Affichage	14
API	14

Ressources

- Pour avoir un retour visuel, j'utilise l'API OpenStreetMap pour la carte et les marqueurs, développé en JavaScript.
- L'application web qui contient la carte est faite en HTML. Cette application permet de charger un json et de l'afficher sur une carte.
- Les programmes pour trier les antennes et rechercher les zones blanches sont faits en C++.
- Le retour du programme de recherche est un Json qui contient, l'itinéraire, les zones blanches, les antennes de couvertures et les polygones des zones blanches.
- Génération d'itinéraires : serveur Valhalla.
- Coordonnées à partir d'une adresse : API OSM Nominatim.
- Points pour les routes : base de données PostgreSQL.
- Fichiers :
 - ABR.cpp | ABR.h
 - dead_zone.cpp | dead_zone.h
 - search_antenne.cpp | search_antenne.h
 - structures.cpp | structures.h
 - find_deadzone.cpp
 - functions.js
 - index.html
 - deadzone.php
- Bibliothèques C++
 - Jsoncpp (parsing JSON)
 - Curl (requêtes vers API)
 - pqxx (gestion PSQL)
 - pugixml (parsing XML)
- Repertoires

```
└─ avance/
   └─ misc/
      └─ data/
         └─ antennes.csv
         └─ antennes/
         └─ ubr/
         └─ json/
            └─ ...
      └─ Visualizer/
         └─ index.html
         └─ icons/
```

Changements

Pour changer les liens vers les répertoires, voir le fichier « **structures.h** ».

Pour modifier les filtres des antennes, voir le fichier « **search_antenne.cpp** ».

Pour changer les informations de la base ou du compte PostgreSQL, voir le fichier « **structures.h** ».

Requête SQL au serveur PostgreSQL, voir le fichier « **structures.cpp** ».

Pour modifier les distances moyennes des antennes selon leur fréquence, voir le fichier « **structures.h** ».

Collecte des antennes

Les outils déjà existants pour avoir une idée du projet :

- Carte des antennes (couverture) : <https://www.zoneadsl.com/couverture-mobile/carte>
- Article technique sur les antennes : <https://blog.juliendelmas.fr/?les-relais-de-telephonie-mobile>
- Carte des antennes 2 : <https://www.cartoradio.fr/index.html#/cartographie/>
- Couverture réseau des routes : <https://www.nperf.com/fr/map/FR/-/13642.Bouygues-Mobile/signal/>
- Base de données à jour (ANFR) : https://data.anfr.fr/visualisation/information/?id=observatoire_2g_3g_4g

Pour obtenir les antennes j'utilise la base de données de l'ANFR, contenant toutes les antennes de France avec une mise à jour mensuelle. De cette base de données j'ai pu extraire uniquement les antennes qui allaient m'être utile, soit celle du type 5G, cela dans un fichier téléchargeable au format CSV.

Données

Coordonnées	Latitude Longitude
Fréquence d'émission (MHz)	700 2100 3500
Support id	« sup_id » identifiant unique de chaque antenne
Opérateurs	SFR, Bouygues, Free, Orange, etc
Département	Numérique : 51, 52, 75, etc
Statut	"Techniquement operationnel" "Projet approuvé" (non fonctionnel)

La table comporte d'autres informations qui ne sont pas utilisées par les programmes.

Une antenne peut être un point de plusieurs émissions, car plusieurs opérateurs peuvent être sur une même structure, nous avons donc une entrée dans la bdd CSV pour chacun d'entre eux toujours avec le même sup_id. Mais nous verrons le tri par la suite.

Parmi toutes ces informations il nous manque la puissance qu'émet une antenne, cela nous permettrait de savoir quel est son rayon de couverture précisément. Avec une formule comme la PIRE.

Actuellement le programme peut s'exécuter avec des estimations car selon la fréquence d'émission, les antennes sont placées à différents endroits stratégiques, voir le tableau ci-dessous.

Informations non sourcées (moyennes) :

Fréquence (MHz)	Distance min (m)	Distance max (m)
700 (rural)	2000	8000
2100 (polyvalent)	1200	4600
3500 (ville)	400	1200

Une autre option est laissée à l'utilisateur, qui est de choisir la portée de toutes les antennes (en m) ou de modifier dans le programme les distances moyennes de chaque fréquence. (cf. [CHANGEMENTS](#)).

Analyse des antennes

Après téléchargement du fichier CSV, on peut commencer le travail d'analyse.

generer_antenne_file

Dans cette fonction on peut appliquer des filtres sur la recherche des antennes grâce aux listes suivantes: « *departement*, *fournisseurs* et *type_5G* » avec en plus le paramètre « *hors_service* » et « *doublon* ». Pour l'instant ces filtres ne sont pas utilisés, on récupère toutes les antennes sauf les doublons et HS.

On ouvre les fichiers en écriture pour le JSON et en lecture pour le CSV.

Parcours du fichier CSV pour récupérer toutes les lignes avec la fonction **getline**. C'est dans cette boucle on retrouve les « if » qui appliques les filtres.

Si c'est la 1^{ère} fois qu'on voit le sup_id d'une antenne on le stock dans un ABR et on le garde en variable comme étant le sup_id du groupe d'antenne (main_sup_id) portant le même sup_id, ainsi dans ce groupe on peut rechercher l'émetteur qui a le rayon le plus grand. Ce qui fait qu'on parcourt toutes les antennes.

À chaque tour de boucle si l'antenne n'a pas été ajoutée, on l'ajoute à la suite dans notre objet json_antenne pour être placé dans un fichier.

readCSVLineAntenne

A chaque ligne du CSV, on appelle cette fonction qui place la ligne dans une structure (AntennaData) pour pouvoir garder les informations qui nous intéressent. La variable de cette structure se nomme « entry ».

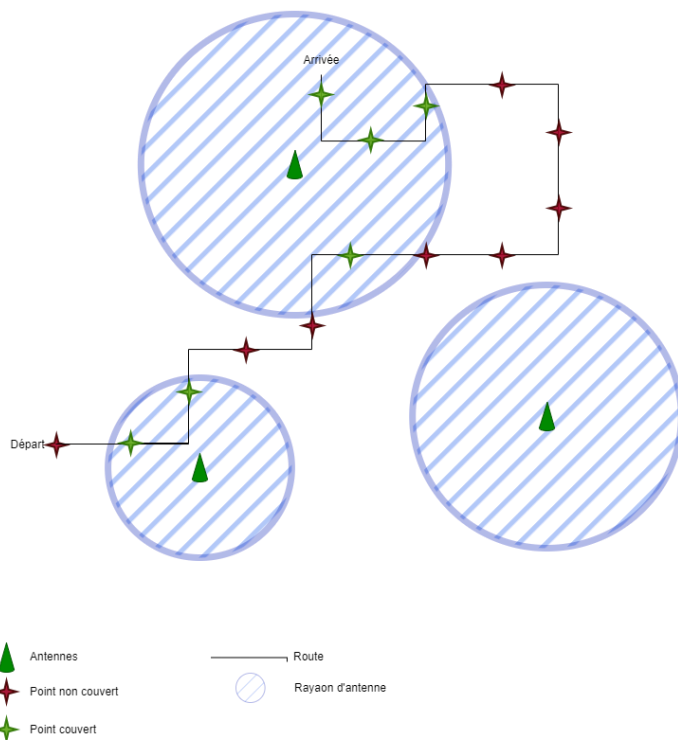
Recherches des zones blanches

Vocabulaire

Dans la suite du CR ou dans le code, une zone blanche peut aussi être nommée « dead_zone » ou « dead_area ». On considère une zone blanche comme un point sur une carte et non un tronçon ou une route, c'est le regroupement de ces points qui va permettre de faire une route.

Algorithme

Pour commencer à aborder ce sujet, j'ai estimé que le moyen le plus simple pour atteindre l'objectif final était de placer des points à intervalles réguliers sur un itinéraire et de vérifier la distance entre les points et les antennes, ce qui pourrait être représenté comme sur le schéma ci-dessous.



Si la distance entre le point et l'antenne est inférieure au rayon d'action de l'antenne, alors ce point est couvert. La précision augmente avec le nombre de points sur l'itinéraire, mais le temps de calcul est affecté.

Version simplifiée

Cette version consiste en la génération d'un seul trajet entre un point de départ et d'arrivée, puis de trouver toutes les zones blanches sur cet itinéraire.

Itinéraire

Pour générer un itinéraire il nous faut 2 points, pour que ce soit intuitif on prend des villes (ou adresses). Grâce à l'API Nominatim, on peut à partir d'une adresse récupérer les coordonnées GPS (latitude et longitude), ici on se sert de notre serveur pour faire cette requête.

get_itineraire

Avec les coordonnées on appelle cette fonction qui va faire une requête à notre serveur Valhalla qui nous retourne un JSON contenant une polyligne de l'itinéraire de manière compressé. Pour décoder la polyligne on utilise une fonction fournie par Valhalla (cf. **structures.cpp** pour voir l'algo) et ainsi on récupère un JSON contenant un itinéraire très précis. L'algo nous générant trop de points pour notre utilisation j'ai mis une variable « pourcent » qui permet de ne garder qu'un pourcentage de point.

find_dead_zone

Cette fonction prend en paramètre :

- Json_itineraire : l'objet qui contient notre itinéraire
- Json_couverture : l'objet qui va contenir les antennes qui couvrent des points
- Json_polyline : l'objet qui va contenir les polygones des dead_zone
- Le nom du fichier JSON qui contient les antennes
- Le rayon des antennes

Elle parcourt tous les points de l'itinéraire et calcul la distance avec les antennes. À chaque tour l'objet json « **Json_polyline** » est rempli.

Exécution

Se placer dans le répertoire « **simple** », si ce n'est pas fait, compiler avec make. Puis exécuter la commande suivante en remplaçant les valeurs.

Paramètres

- Ville de départ
- Ville d'arrivée
- Fichier d'antennes (json)
- Pourcentage de points, plus il est élevé, plus c'est précis mais long
- Rayon d'antenne (voir : [ANTENNES](#))
- Option sauvegarde (voir : [OPTIONS](#))

```
./MAIN REIMS PARIS ANTENNE_FRANCE.JSON 10.0 1.2 0
```

Version avancée

Cette version consiste en l'analyse de toutes les routes se trouvant autour d'un point dans un rayon choisi par l'utilisateur.

Itinéraires

Pour obtenir tous les itinéraires, on utilise la base de données PostgreSQL qui est présente sur le serveur. Pour comprendre son fonctionnement, voir le manuel de [NoahG](#). Pour le mot de passe du compte postgres voir M. Fouchal. **Si le mot de passe est changé, voir [CHANGEMENTS](#).**

Dans cette base on retrouve plusieurs tables avec le préfixe « planet_osm_ ».

Celle que l'on utilise est la table « planet_osm_line » elle comporte tous les tronçons de route possibles. Ces tronçons ne font pas toujours toute la longueur de la route.

La colonne qui nous intéresse est « way » elle est dans un format Geopoint, avec le formatage « ST_AsKML(way) » on récupère un tableau de coordonnées au format XML qu'on analyse comme un json avec la bibliothèque appropriée. On utilise également la colonne « highway » qui va nous permettre de filtrer les tronçons.

Table planet_osm_line: highway

Cette colonne va nous permettre de ne garder que les routes qui nous intéressent et d'éliminer les routes maritimes, chemin de terre, voie pédestre, etc.

Routes (highway)

- Trunk / trunk link : nationale
- Residential : ville et village
- Unclassified : autres
- Motorway / motorway link : autoroute
- Primary / primary link : grands axes
- Secondary / secondary link : departmental
- Tertiary / tertiary link : departmental

Pour changer le filtre voir [CHANGEMENTS](#).

À la réception des données de la requête (~2 min), on a donc plusieurs tableaux de petits itinéraires que l'on va placer dans un objet json. Seuls les points se trouvant dans le rayon choisi sont ajoutés. On utilise la fonction **parse_line** pour analyser chaque ligne de la table.

get_lines

Cette fonction récupère les routes dans la base de données.

Elle prend en paramètre une latitude, longitude, le rayon du cercle, la distance entre chaque point et un objet json en référence qui va contenir tous les itinéraires.

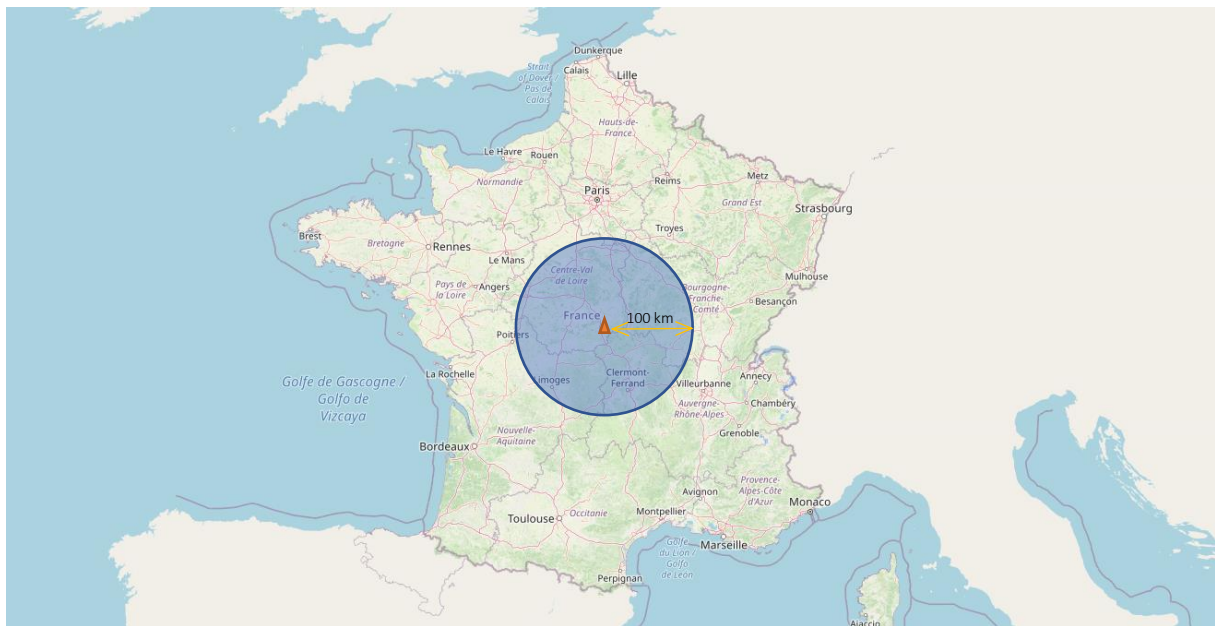
Zones blanches

Surface de recherche

Pour faire la corrélation entre les antennes et les routes, on place un point sur la carte et on effectue les calculs dans un rayon donné par l'utilisateur.

Le programme va lancer la fonction « **find_dead_zone** » qui exécute un nombre de thread égal au nombre de cœur de la machine. Chaque thread a une partie des itinéraires à traiter, c'est une double boucle « for » qui va faire passer chaque itinéraire puis dans chacun d'entre eux faire le calcul entre chaque point de l'itinéraire et les antennes. Un affichage en console, indique le nombre de thread créé et la position de départ et d'arrivée dans le tableau d'itinéraires pour chacun d'entre eux.

Par convention et sécurité on met des mutex sur les objets json qui vont être modifiés à savoir, la polyline, la deadzone, la couverture et l'ABR de sup_id.



En commande on va donc mettre la **latitude** et **longitude** du centre, puis le **rayon du cercle** par rapport au centre (valeur en m).

Antennes

Le fait que nous n'avons pas le rayon d'action précis des antennes, le programme propose 3 solutions pour faire les calculs.

1. Utilisation d'approximations moyennes maximales ([DONNÉES](#)) : on applique le rayon maximum théorique moyen que peut avoir l'antenne. Valeur « 0 » dans la commande.
2. Utilisation d'approximations moyennes minimums ([DONNÉES](#)) : on applique le rayon minimum théorique moyen que peut avoir l'antenne. Valeur « -1 » dans la commande.
3. Choix utilisateur : l'utilisateur choisit la distance que toutes les antennes vont avoir. Valeur « >0 », en m.

UBR

C'est le même principe que pour les antennes, on considère les calculs avec des « antennes » artificielles qui vont être des points UBR. S'il y a un fichier CSV, il doit être placé dans le répertoire **misc/data/ubr/** et on met le fichier en commande sinon on met « null ».

Pour un bon usage, le fichier CSV ne doit contenir que 4 colonnes dans cet ordre : « latitude ; longitude ; rayon ; état » avec une ligne d'en-tête. Pour que le programme prenne en charge toutes les données il faut veiller à retirer les caractères spéciaux comme le « ZWNBSP » qui est invisible mais entrave la conversion des données. Il faut également éviter de garder des espaces vides après les données.

Exemple de fichier csv :

	A	B	C	D
1	Latitude	Longitude	Rayon	Etat
2	47,267838	-1,572669	3000	en service
3	47,2711222	-1,557655	3000	en service
4	48,312387	-1,34644	3000	en service
5	48,357352	-2,313445	3000	en service
6	48,442148	-2,478245	3000	en service
7	48,170568	-1,894997	3000	en service
8	48,207994	-2,054118	3000	en service
9	48,139576	-1,698061	3000	en service
10	47,8322194	-1,6919333	3000	en service
11	47,764744	-1,710303	3000	en service
12	47,3360833	-1,6536917	3000	en service
13	47,553947	-1,632326	3000	en service
14	47,6717694	-1,68805	3000	en service
15	47,38261	-1,967315	3000	en service
16	47,4468611	-2,1196666	3000	en service
17	47,5173584	-1,6861167	3000	en service
18	47,3867972	-1,887975	3000	en service
19	47,3489945	-2,0682389	3000	en service
20	47,324384	-2,151711	3000	en service
21	47,218107	-1,477025	3000	en service

Routes

Ce paramètre permet de choisir sur un itinéraire, la distance entre chaque point. Plus la distance est grande, moins il y aura de points donc le temps de calcul est réduit, mais la précision est également réduite.

Les tests ont été effectués avec une distance de 500 mètres entre chaque point. Il se peut que 2 points ne respectent pas cette distance, cela signifie qu'ils font partie de 2 tronçons différents.

Option de sauvegarde

Cette option permet de soit afficher en console (valeur « 1 ») avec un « `cout << json_str << endl` », utilisé pour l'API.

Sinon de sauvegarder le json dans un fichier à cet [EMPLACEMENT](#) (valeur « 0 »).

Fichier de retour

Le fichier de retour ou l'affichage console est un json dans avec ces attributs : `itineraire`, `dead_zone`, `couverture` et `polyline`.

Les polygones sont présentes pour avoir un retour visuel des zones blanches, pour avoir les zones blanches individuellement point par point, cela se trouve dans « `dead_zone` », mais ça ne forme pas d'itinéraire.

- **Itineraire** : ce sont tous les itinéraires utilisés.
 - `Dept`
 - `Id`
 - `Lat`
 - `Lon`
- **Dead_zone** : ce sont tous les points non couverts par une antenne.
 - `Dept`
 - `Id`
 - `Lat`
 - `Lon`
- **Couverture** : ce sont toutes les antennes qui couvrent les points des itinéraires.
 - `Dept`
 - `Id`
 - `Lat`
 - `Lon`
 - `Rayon_moy_max`
 - `Rayon_moy_min`
 - `Rayon_utilise`
 - `Operateur`
- **Polyline** : ce sont toutes les polygones (tronçons de routes) qui ne sont pas couverts.
 - `Lat`
 - `Lon`
- **Ubr** : ce sont tous les points ubr mit sur la carte.
 - `Lat`
 - `Lon`
 - `Rayon`
 - `Etat`

Système d'affichage web

Pour avoir un retour visuel du fichier généré, on utilise une carte d'OpenStreetMap. Elle nous permet de voir les routes sous forme de polygones et de points, de même pour les routes non couvertes et de voir les antennes qui couvrent les routes.

L'affichage de la carte est fait en javascript avec la fonction « **afficher_carte** », la carte utilisée est celle du serveur OSM du Lab-I*.

Pour le filtre sur la carte, ce sont des checkbox déguisés en boutons qui permettent de choisir ce que l'on affiche. À chaque appui, cela appelle la fonction « **afficher_carte** », celle-ci selon les checkbox qui sont cochés va générer ou non les marqueurs et groupes de marqueurs pour les ajouter à la carte. La fonction traite donc également le fichier json qui est chargé

Manuel d'utilisation version avancée

Pour cette partie du projet nous avons 2 programmes : **search_antenne** qui permet de trier les antennes et **find_deadzone** qui fait la corrélation entre notre fichier d'antennes et la bdd de routes. On se trouve dans le répertoire « avance ».

Récupération des antennes

Téléchargement des données

Aller télécharger le fichier CSV des antennes sur le site de l'ANFR :

https://data.anfr.fr/visualisation/export/?id=observatoire_2g_3g_4g&refine.generation=5G

Il est mieux de trier les antennes à ce moment en ne sélectionnant que 5G dans « Technologies » dans la zone de « filtres actifs », sinon le programme va être plus long.

Données sur les réseaux mobiles

The screenshot shows the ANFR data export interface. On the left, under 'Filtres actifs', 'Technologie' is set to '5G' and '96 471 enregistrements' are displayed. Below this, a table lists operators: BOUYGUES TELECOM (27 687), FREE MOBILE (31 866), ORANGE (13 788), and OUI FREEMER TELECOM (75). On the right, the 'Export' tab is selected, showing 'Formats de fichiers plats' with options for CSV, JSON, and Excel. For each format, there are two download links: 'Jeu de données entier' and 'Seulement les 96471 enregistrements sélectionnés'. The 'Seulement les 96471 enregistrements sélectionnés' link for CSV is highlighted with an orange box. A note at the top right states 'Ce jeu de données est sous licence : Licence Ouverte v2.0 (Etalab)'.

Emplacement du fichier

Placer le fichier CSV dans le répertoire **misc/data/** afin qu'il puisse être lu par le programme.

Exécution

Ce placer dans le répertoire « avance » ou source s'il est renommé. Dans ce répertoire il faut que l'exécutable « **search_antenne** » soit présent.

Si ce n'est pas fait, compiler avec *make*, ne pas hésiter à faire un *make clean* avant pour être sûr que tous les fichiers soient bien compilés.

Pour l'exemple le fichier CSV que je vais utiliser, se nomme « *antennes_avril_2024.csv* », normalement le nom original du fichier au téléchargement est « *observatoire_2g_3g_4g.csv* ».

Quant au fichier créé qui va contenir les antennes triées, il va être créé à l'emplacement : **misc/data/antennes/**. Nous le nommerons pour l'exemple « *antenne_france.json* ». (cf. partie [CHANGEMENTS](#) pour modifier le lien).

Toujours dans le même répertoire on peut exécuter le programme c++ en plaçant en paramètre les noms des fichiers sans les extensions :

```
./SEARCH_ANTENNE ANTENNE_FRANCE ANTENNES_AVRIL_2024
```

L'exécution se termine correctement par la phrase « Toutes les antennes ont été sauvegardés ».

Ce json est un tableau qui contient les antennes, on ne garde que le département (dept), la latitude (lat), la longitude (lon), support id (sup_id), le rayon moyen minimum (rayon_moy_min), le rayon moyen maximum (rayon_moy_max) et l'opérateur (opérateur).

Trouver les zones blanches

Avant l'exécution il faut impérativement que le fichier json qui contient les antennes se trouve dans le répertoire **misc/data/antennes/** ou dans celui qui est indiqué dans le fichier « **structures.h** ».

Emplacement du fichier

Le fichier json généré se trouve dans le répertoire **misc/data/json/** et se nomme « **json_line_[rayon]_[antenne rayon]_[ubr load].json** ». (cf. partie [CHANGEMENTS](#) pour modifier le lien).

Exécution

Nous reprendrons pour l'exemple le fichier d'antennes de l'exemple précédent, donc « **antenne_france.json** ».

Ce placer dans le répertoire « **avance** » ou source s'il est renommé. Dans ce répertoire il faut que l'exécutable « **find_deadzone** » soit présent.

Si ce n'est pas fait, compiler avec *make*, ne pas hésiter à faire un *make clean* avant pour être sûr que tous les fichiers soient bien compilés.

Pour l'exécution il va falloir les coordonnées GPS du centre du cercle qui va couvrir les routes donc latitude et longitude, le nom du fichier des antennes générés « **antenne_france.json** », le rayon en mètre du cercle, le rayon des antennes, la distance entre 2 points sur un itinéraire, l'option (sauvegarde en fichier ou affichage console) du nouveau json généré et du fichier UBR ou non. (voir : [ZONE BLANCHES](#) pour comprendre les paramètres).

Il est préconisé d'utiliser la commande *nohup* pour une grande surface. *100 km de rayon avec des antennes qui couvre le rayon maximum, correspond à ~30 min de calcul.*

Pour l'exemple on se place au centre de la France avec 100 km de rayon, les antennes on le rayon maximum et des points tous les 500 mètres. Le 1^{er} utilise un fichier ubr, le 2^{ème} non.

```
./FIND_DEADZONE 46.33417962951923 2.517764534293816 ANTENNE_FRANCE.JSON  
100000 0 500 0 UBR.CSV
```

```
./FIND_DEADZONE 46.33417962951923 2.517764534293816 ANTENNE_FRANCE.JSON  
100000 0 500 0 NULL
```

Si la récolte de la base de données s'est faite correctement, un message de succès avec le timestamp et le temps écoulé depuis le départ devrait s'afficher.

Ex : « BDD entièrement fetch : 1715783659482, tps :130505ms »

L'exécution complète du programme se termine par le timestamp et le temps total d'exécution [h:m:s:ms].

Ex : « 0h : 2m : 50s : 317ms ».

Affichage

Pour afficher une carte avec les zones blanches, aller dans le répertoire « **./Visualizer** » et ouvrir le fichier « **index.html** ». Pour plus d'informations sur le programme, voir [SYSTEME AFFICHAGE](#).

Sur la page on peut charger le fichier json généré ou charger le fichier contenant les antennes.

Chaque bouton permet d'afficher ou non certaine partie de la carte.

- **Point dead zone** : affiche tous les points qui ne sont pas couverts par une antenne.
- **Antenne** : affiche toutes les antennes qui couvrent des points, les points wifi aussi.
- **Itinéraire** : affiche tous les points composant les itinéraires.*
- **Polyline dead zone** : affiche tous les tronçons de route qui ne sont pas couverts.
- **Polyline route** : affiche tous les tronçons de route.*

* ! Si le fichier comporte trop de point, la mémoire du navigateur peut être saturé !

API

Si le programme doit être exécuté autrement qu'avec une console, on peut faire une requête GET à un fichier PHP « **deadzone.php** » qui va exécuter le programme et retourner le json généré.

Il faut que le fichier se trouve dans le même répertoire que le programme compilé « **find_deadzone** » et que les fichiers à compiler soient dans fait dans le même OS que le serveur. Pour l'instant tout est configuré pour tourner sur un linux.

Si le fichier « **deadzone.php** » doit changer d'emplacement, il faudra également changer le lien vers l'exécutable et le lien vers le fichier contenant les antennes dans la fonction « **exec** ».

Requête

[http://localhost/avance/deadzone.php?lat_centre={lat}&lon_centre={lon}&r_centre={rayon_centre \(m\)}&rayon={rayon_antenne \(m\)}&dist={dist_route \(m\)}](http://localhost/avance/deadzone.php?lat_centre={lat}&lon_centre={lon}&r_centre={rayon_centre (m)}&rayon={rayon_antenne (m)}&dist={dist_route (m)})